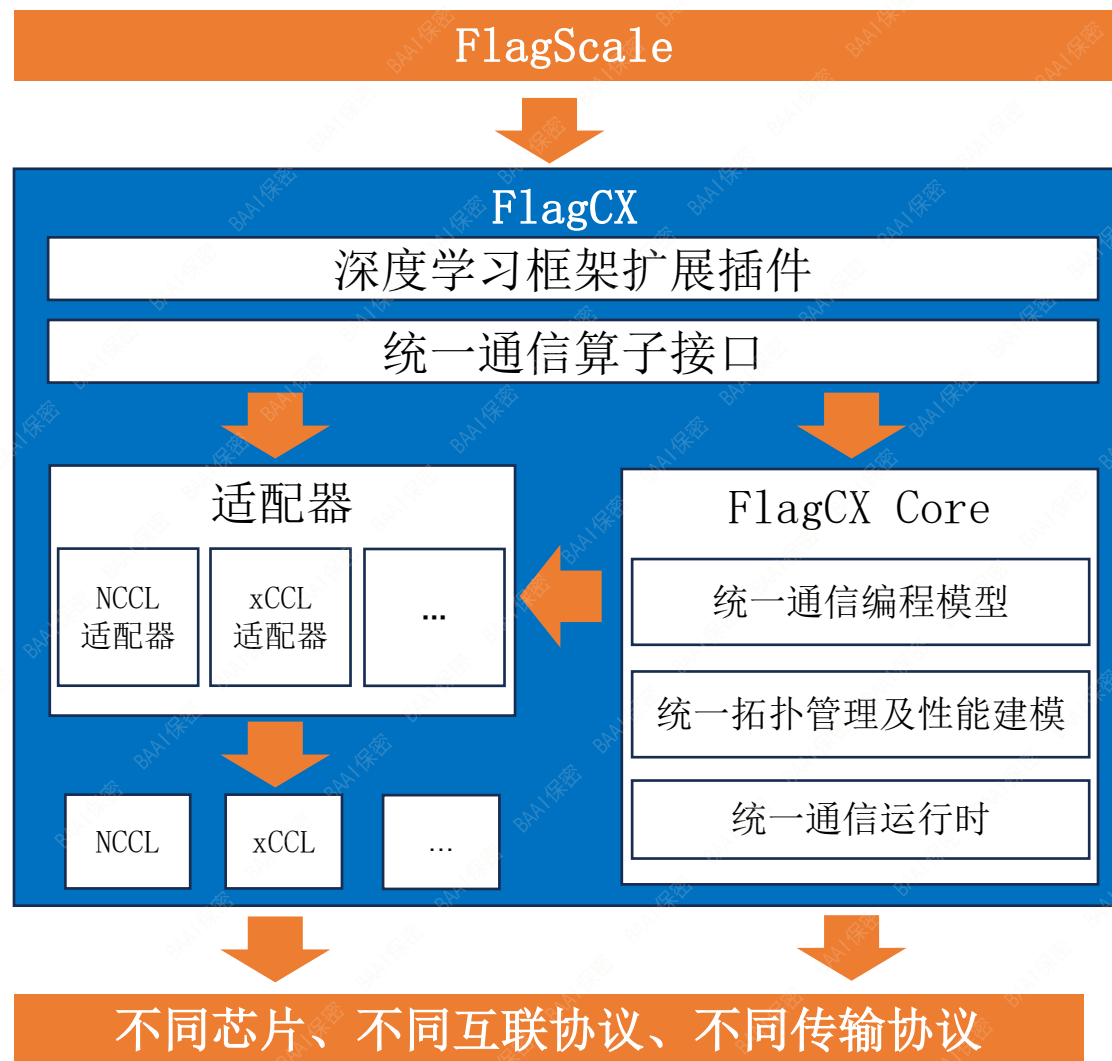


*FlagCX*跨芯通信

Cross-chip Communication

目录

- 1 FlagCX背景介绍
- 2 Adaptor设计范式
- 3 Device-buffer RDMA技术详解
- 4 Cluster-to-Cluster算法



开源地址: <https://github.com/FlagOpen/FlagCX>

设计目标

- › 实现同一芯片大规高效稳定通信（容错、故障检测）
- › 实现不同芯片高效通信，支持跨芯高效异构混训
- › 实现基于统一编程模型的异构集合通信算法编译模拟

最新特性

- › 支持8+3种原生通信库和自动拓扑探测功能
- › 支持11种异构集合通信算法，适用多网卡/单网卡等多种复杂场景
- › 提供FlagScale + FlagCX高效异构混训全栈开源方案；支持百度飞桨+FlagCX跨芯通信；原生集成浪潮元脑平台

当前已支持功能

通信操作支持

(all)reduce
(all)gather
(reduce)scatter
alltoall(v)
send/recv
broadcast

适配器支持

nvidia
iluvatar corex
cambricon
metax
kunlunxin
du
ascend
musa

框架支持

PyTorch
Paddle

拓扑支持

服务器拓扑感知
最优网卡匹配

CCLAdaptor

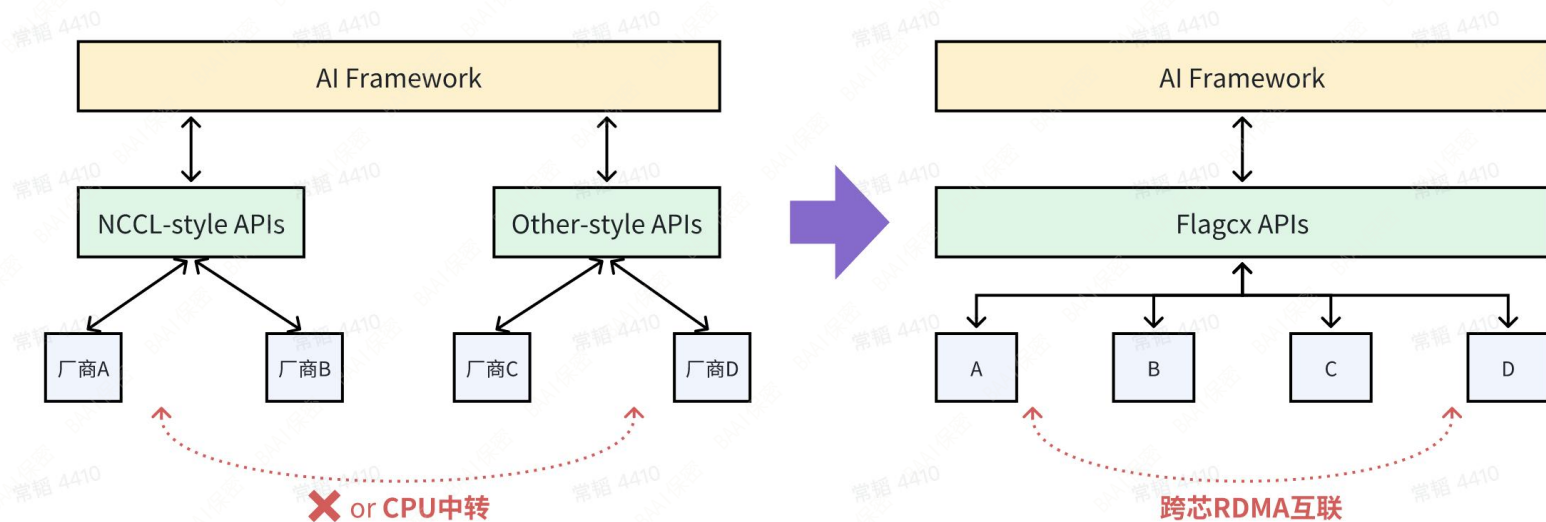
- 负责提供厂商原生通信库抽象 (NCCL、HCCL等)

DeviceAdaptor

- 负责提供厂商原生设备运行时抽象 (CUDA、CANN等)

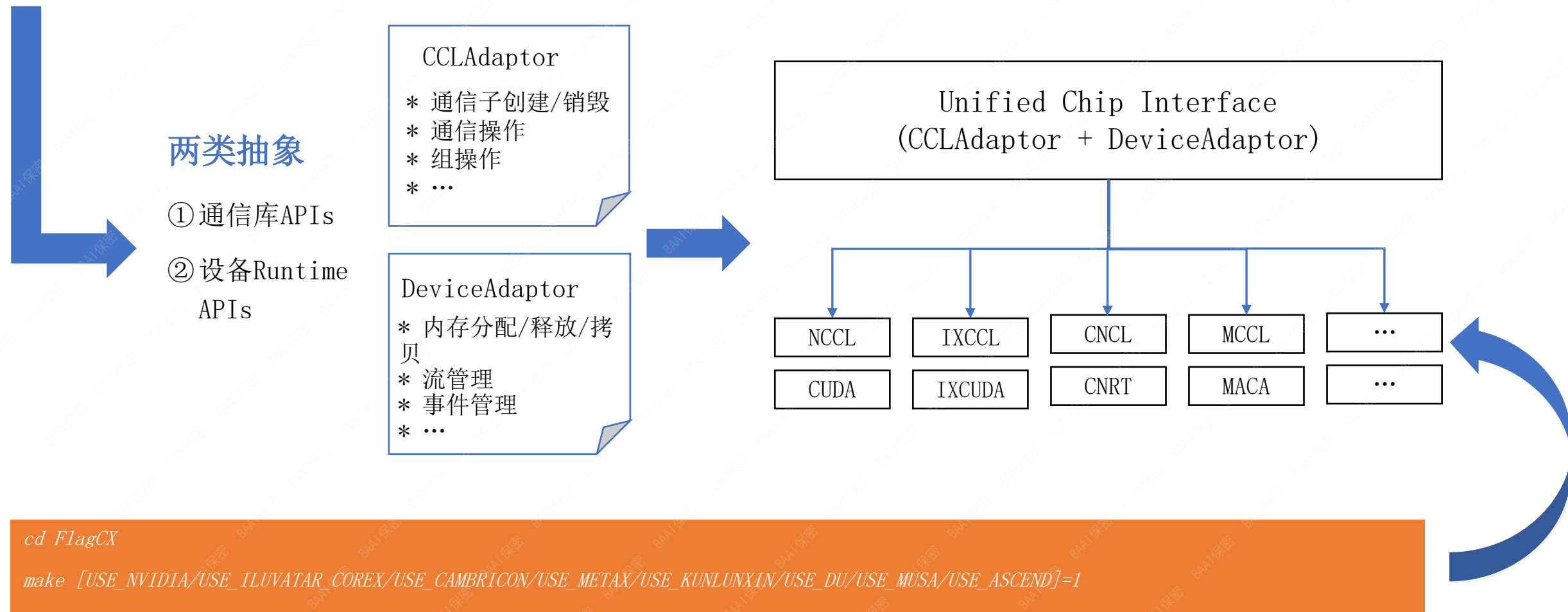
TransportAdaptor

- 负责提供厂商原生传输抽象 (NET IB/RoCE、NET TCP/IP等)



Adaptor – 多后端适配器

目标：支持一键编译实现多种芯片高效同构通信



CCLAdaptor API

```
struct flagcxCCLAdaptor {
    const char name[32];
    // Basic functions
    flagcxResult_t (*getVersion)(int *version);
    flagcxResult_t (*getUniqueld)(flagcxUniqueld_t *uniqueld);
    const char *(*getErrorString)(flagcxResult_t result);
    const char *(*getLastError)(flagcxInnerComm_t comm);

    // Communicator functions
    flagcxResult_t (*commInitRank)(flagcxInnerComm_t *comm, int nranks,
                                   flagcxUniqueld *commId, int rank,
                                   bootstrapState *bootstrap);
    flagcxResult_t (*commDestroy)(flagcxInnerComm_t comm);
    ...
    // Communication functions
    ...
    flagcxResult_t (*allReduce)(const void *sendbuff, void *recvbuff,
                               size_t count, flagcxDataType_t datatype,
                               flagcxRedOp_t op, flagcxInnerComm_t comm,
                               flagcxStream_t stream);
    flagcxResult_t (*recv)(void *recvbuff, size_t count,
                           flagcxDataType_t datatype, int peer,
                           flagcxInnerComm_t comm, flagcxStream_t stream);
    ...

    // Group semantics
    flagcxResult_t (*groupStart)();
    flagcxResult_t (*groupEnd)();
};
```

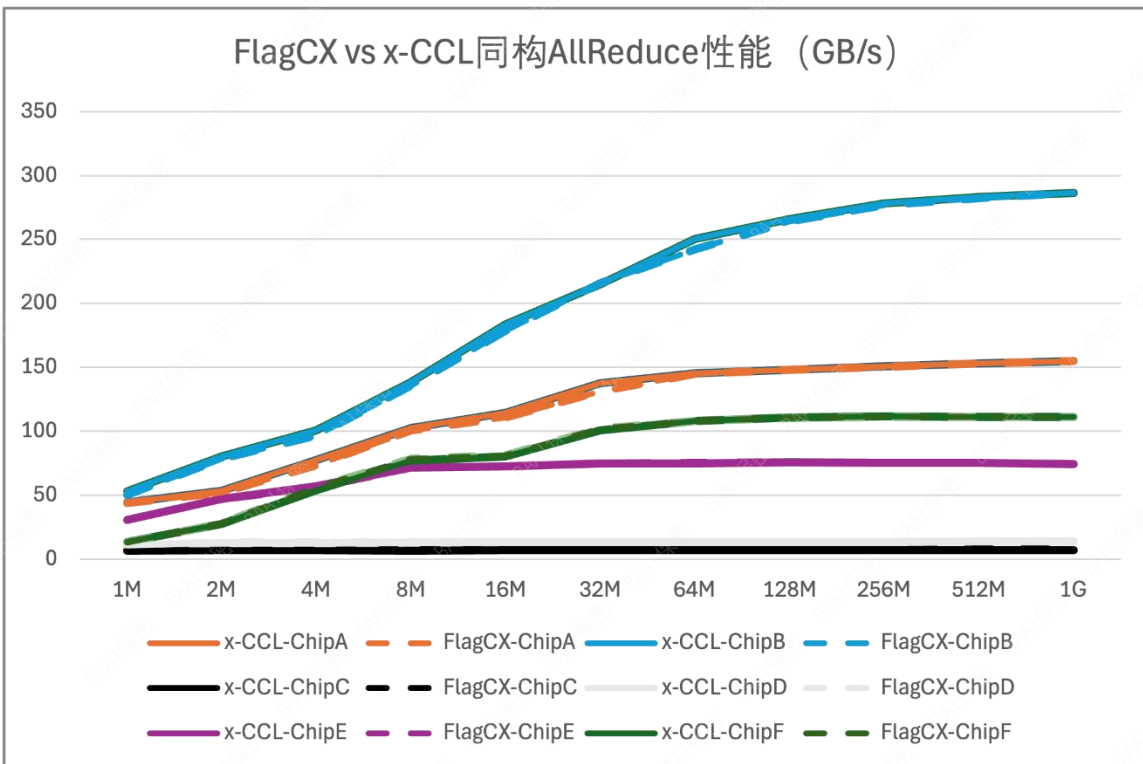
DeviceAdaptor API

```
struct flagcxDeviceAdaptor {
    char name[32];
    // Basic functions
    flagcxResult_t (*deviceSynchronize)();
    flagcxResult_t (*deviceMemcpy)(void *dst, void *src, size_t size,
                                    flagcxMemcpyType_t type, flagcxStream_t stream,
                                    void *args);
    ...
    // GDR functions
    flagcxResult_t (*gdrMemAlloc)(void **ptr, size_t size, void *memHandle);
    flagcxResult_t (*gdrMemFree)(void *ptr, void *memHandle);
    ...
    // Stream functions
    flagcxResult_t (*streamCreate)(flagcxStream_t *stream);
    flagcxResult_t (*streamDestroy)(flagcxStream_t stream);
    ...
    // Event functions
    flagcxResult_t (*eventCreate)(flagcxEvent_t *event);
    flagcxResult_t (*eventDestroy)(flagcxEvent_t event);
    ...
    // Host/DeviceFunc launch
    flagcxResult_t (*launchHostFunc)(flagcxStream_t stream, void (*fn)(void *),
                                     void *args);
    flagcxResult_t (*launchDeviceFunc)(flagcxStream_t stream,
                                       flagcxLaunchFunc_t fn, void *args);
    ...
};
```

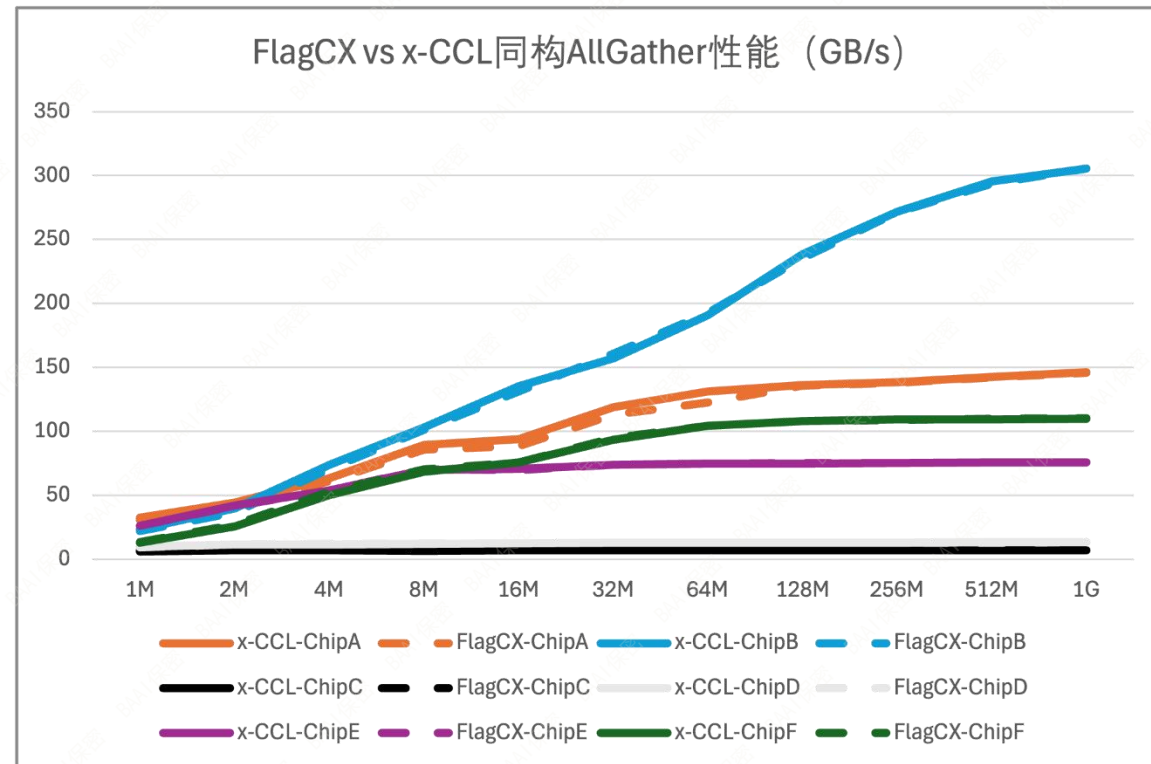

Adaptor – 多后端适配器（性能数据）

同构场景零开销

FlagCX vs x-CCL同构AllReduce性能 (GB/s)



FlagCX vs x-CCL同构AllGather性能 (GB/s)



Adaptor – 跨芯通信

目标：支持多种芯片高效异构（跨芯）通信

三类抽象

- ① 通信库APIs
- ② 设备Runtime APIs
- ③ 网络传输APIs

CCLAdaptor

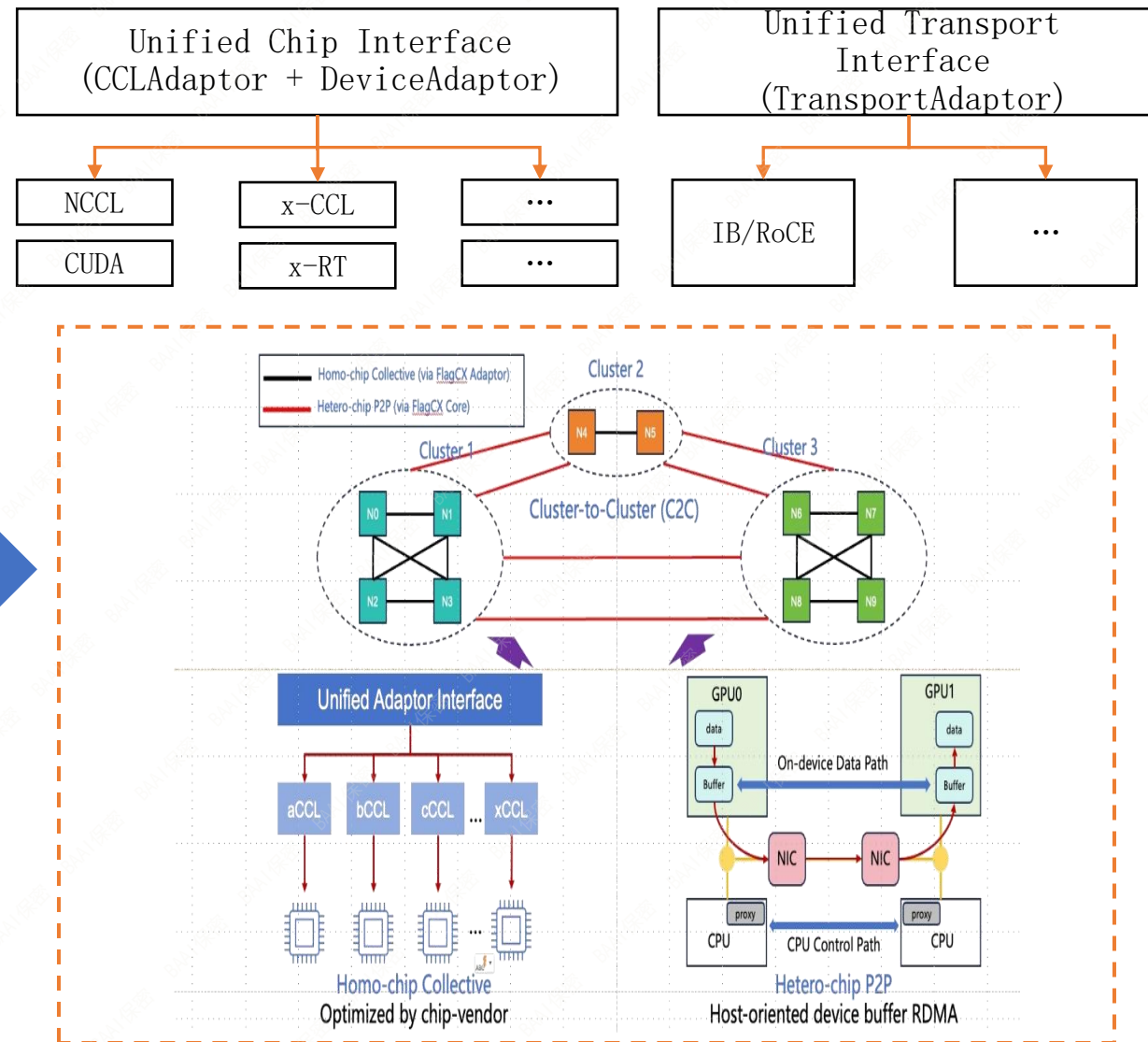
- * 通信子创建/销毁
- * 通信操作
- * 组操作
- * ...

DeviceAdaptor

- * 内存分配/释放/拷贝
- * 流管理
- * 事件管理
- * ...

TransportAdaptor

- * NET IB/RoCE
- * NET TCP/IP
- * ...



Adaptor – 总结

1. CCLAdaptor + DeviceAdaptor + TransportAdaptor

提供**多芯+多传输支持**



高可迁移性

一键编译多芯支持 (8+)

GPU-DirectRDMA

- RDMA基本概念
- GDR技术演进
- GDR执行流程

Device-buffer RDMA

- 执行流程
- 内存注册

基本工作原理：

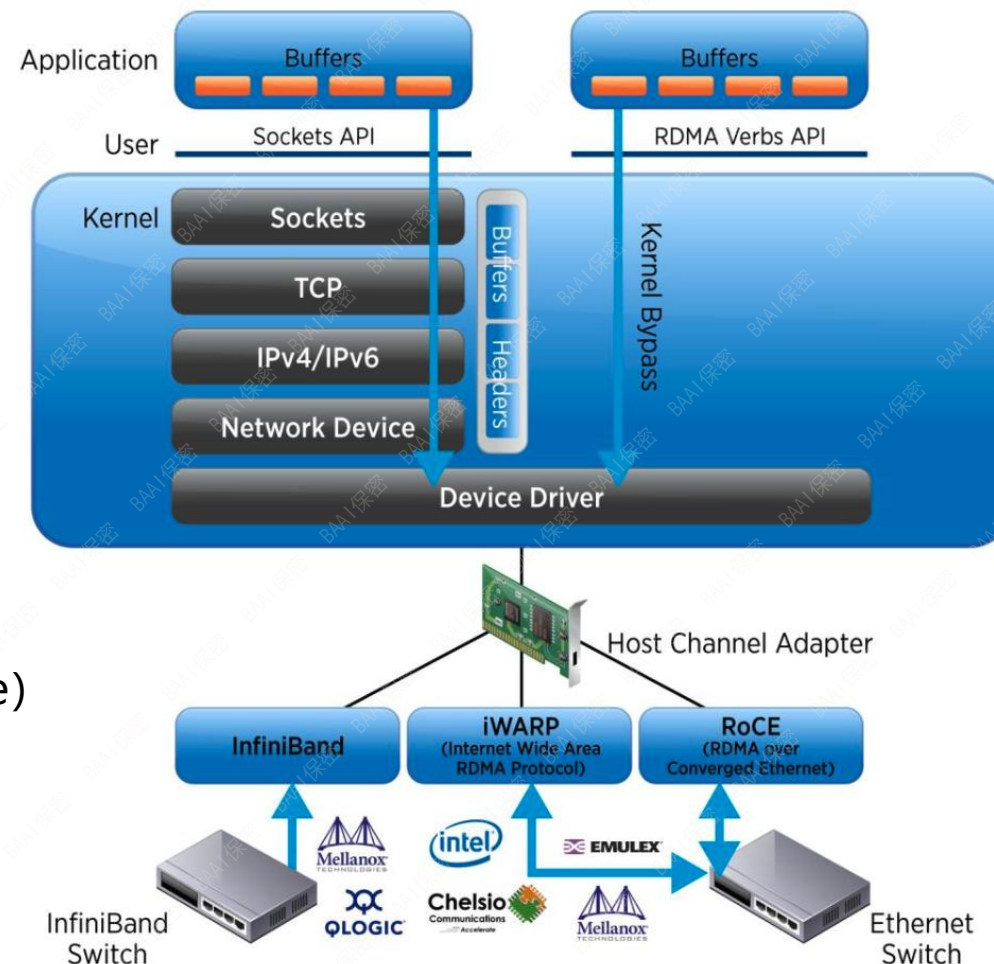
- RDMA 流量直接发送到网卡（NIC），无需中断 CPU
- 远程内存区域需要先在网卡上注册
- 网卡记录虚拟地址到物理页的映射关系
- 通信双方都可以绕过内核进行数据传输

工作队列（WQ）：

- 包含工作请求（WR, Work Request）的队列
- 可以是发送队列或接收队列（SQ/RQ, Send Queue/Receive Queue）
- 发送队列和接收队列始终作为一个对象统一存在，QP（Queue Pair）

完成队列（CQ）：

- 由完成条目（CE, Completion Entry）组成的队列
- 完成条目中包含工作队列（WQ）中某个操作的状态、字节数以及额外的元数据



2022 APAC RDMA Lecture: User-level Networking

RDMA操作:

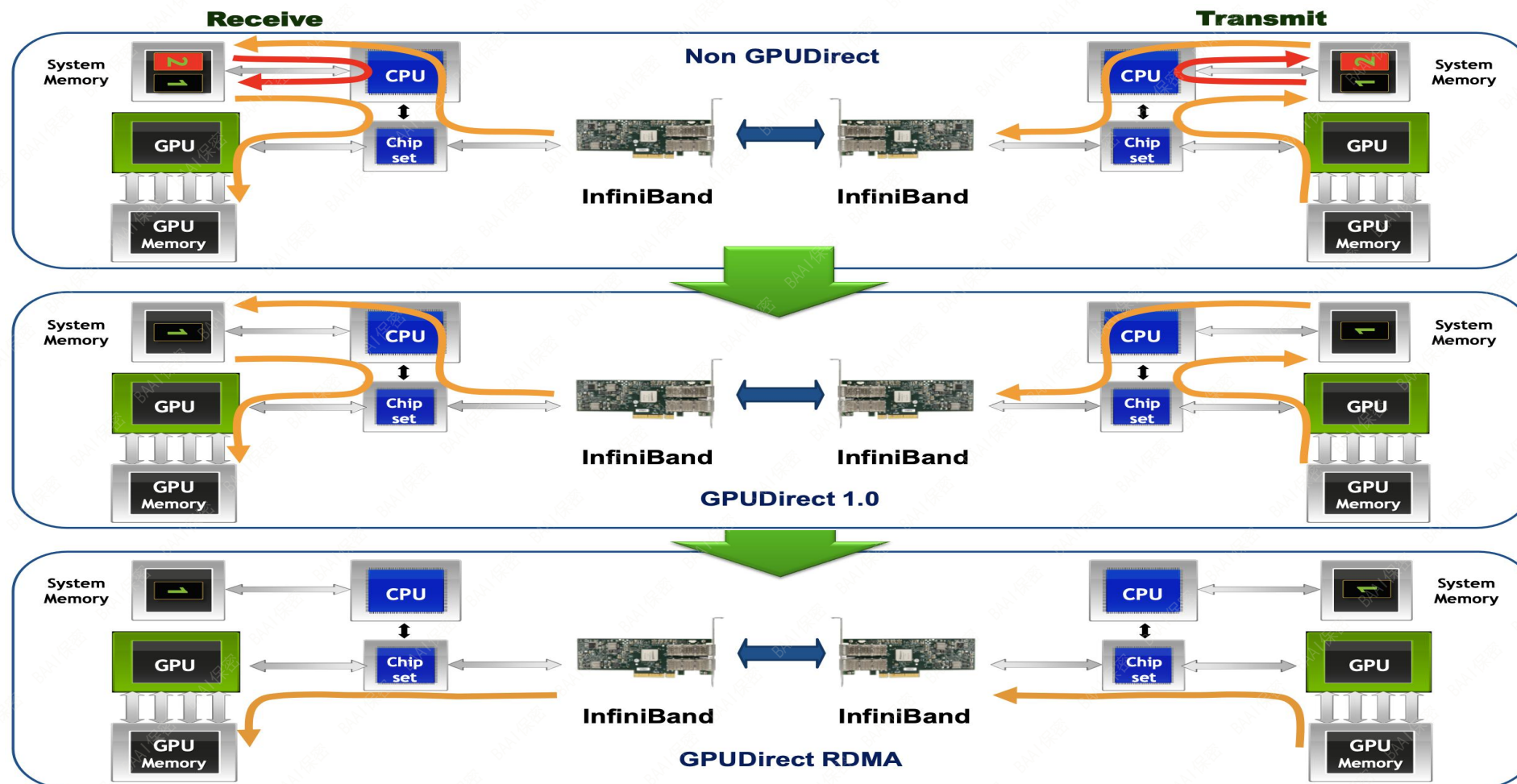
- Send/Receive, 双边通信
- Write, 单边通信
- Read, 单边通信
- Atomic ops, 单边通信

RDMA传输服务:

- RC, reliable transport, connection oriented
- UC, unreliable transport, connection oriented
- UD, unreliable transport, connectionless
- DC, reliable transport, dynamically connected

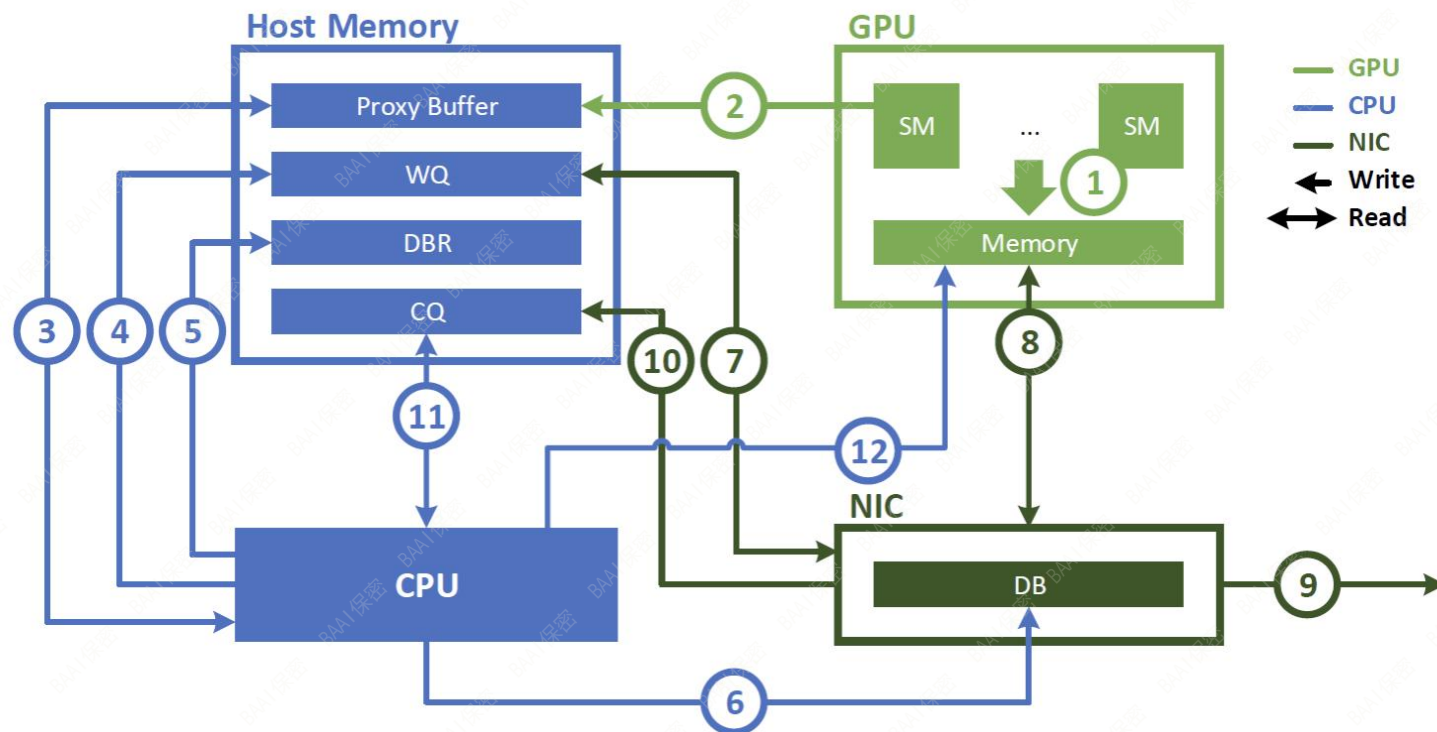
Operation	RC	UC	UD	DC
Send/Receive	☑	☑	☑	☑
Write	☑	☑	✗	☑
Read	☑	✗	✗	☑
Atomics	☑	✗	✗	☑
Reliability	High	Low	Low	High
Scalability	Low	Low	High	Low

Device-buffer RDMA - GDR技术演进



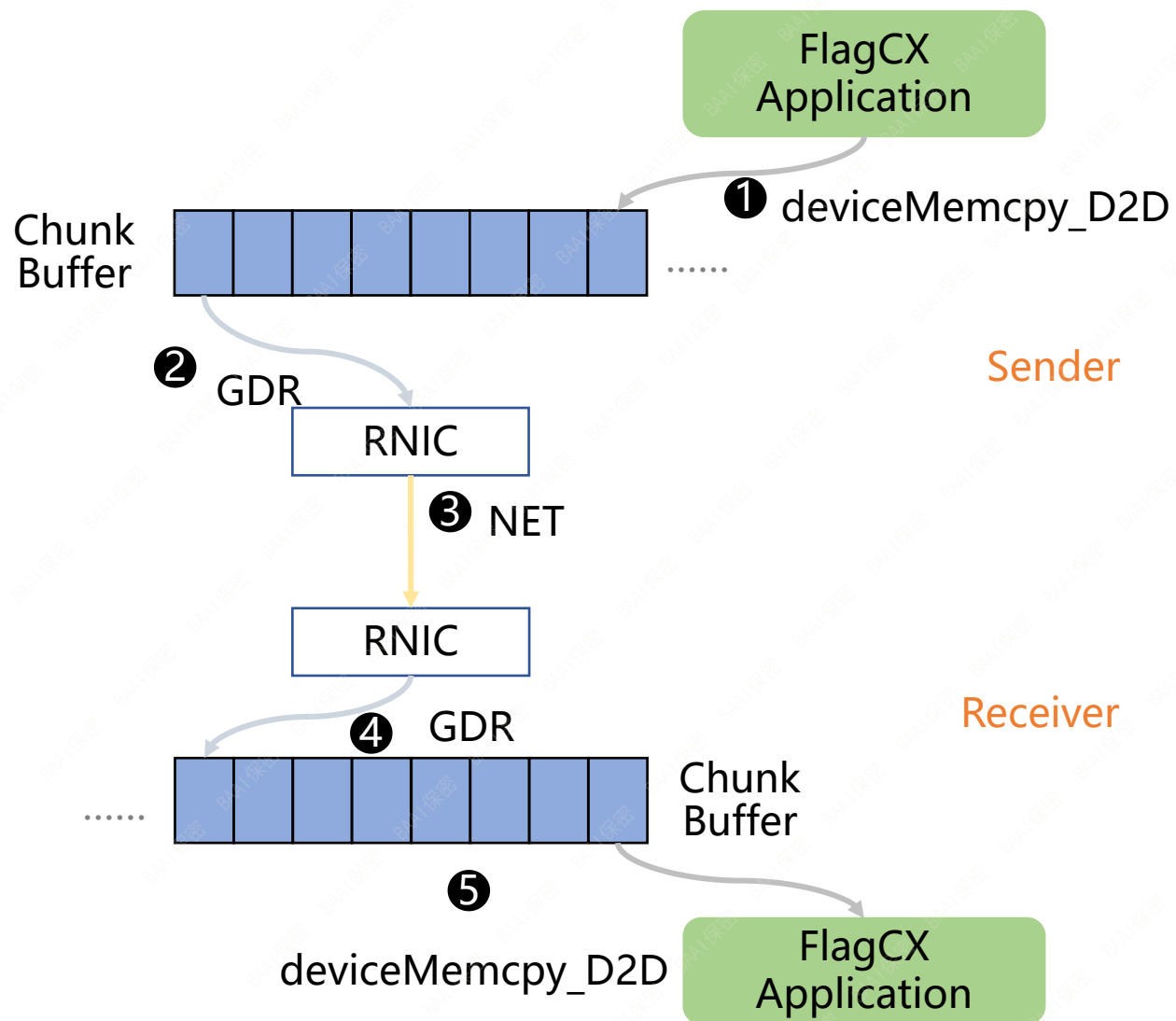
GPU-InfiniBand Accelerations for Hybrid Compute Systems

Device-buffer RDMA – GDR执行流程



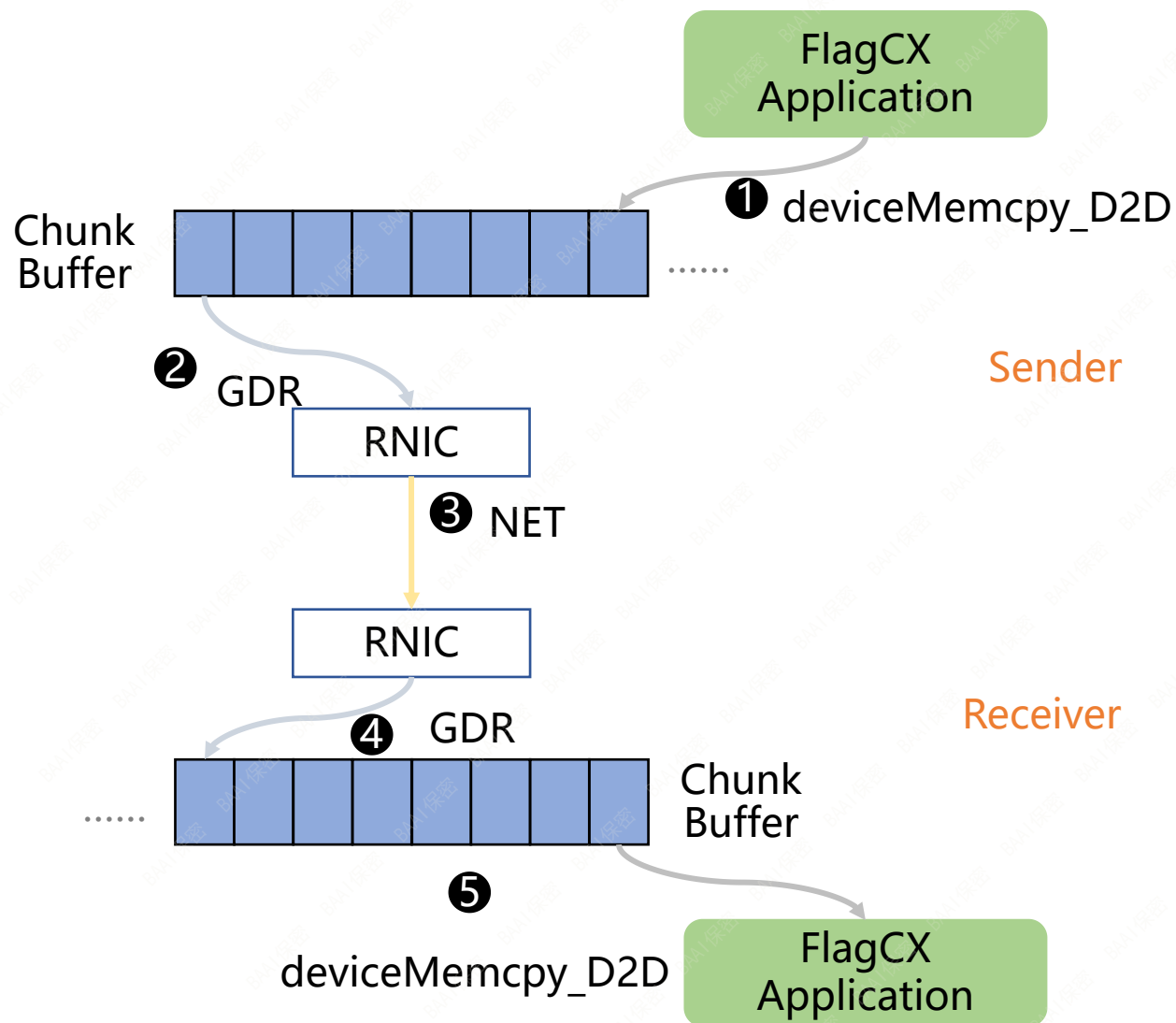
1. 应用程序启动Kernel，在GPU内存中产生数据
2. Kernel将一个work descriptor写入host memory中的proxy buffer
3. CPU从proxy buffer中获取通知
4. CPU将work descriptor加入WQ buffer
5. CPU更新host memory中的DBR buffer，用于doorbell signal recovery
6. CPU通过写入NIC的DB通知NIC。DB是NIC硬件中的一个寄存器
7. NIC从WQ buffer中读取work descriptor
8. NIC直接从GPU内存搬运数据
9. NIC将数据传输到远程节点
10. NIC更新host memory中的CQ buffer
11. CPU轮询CQ buffer以检测网络操作的完成
12. CPU通知GPU网络操作已完成

Device-buffer RDMA – 执行流程



1. 发送方：将源数据从应用Buffer拷贝到Chunk Buffer
2. 发送方：RNIC通过GDR从Chunk Buffer读取数据，发送至网络
3. TransportAdaptor接口实现网络传输
4. 接收方：从网络接收数据，并将接收到的数据通过GDR存入Chunk Buffer
5. 接收方：将数据从Chunk Buffer拷贝至目标应用Buffer

Device-buffer RDMA – 执行流程



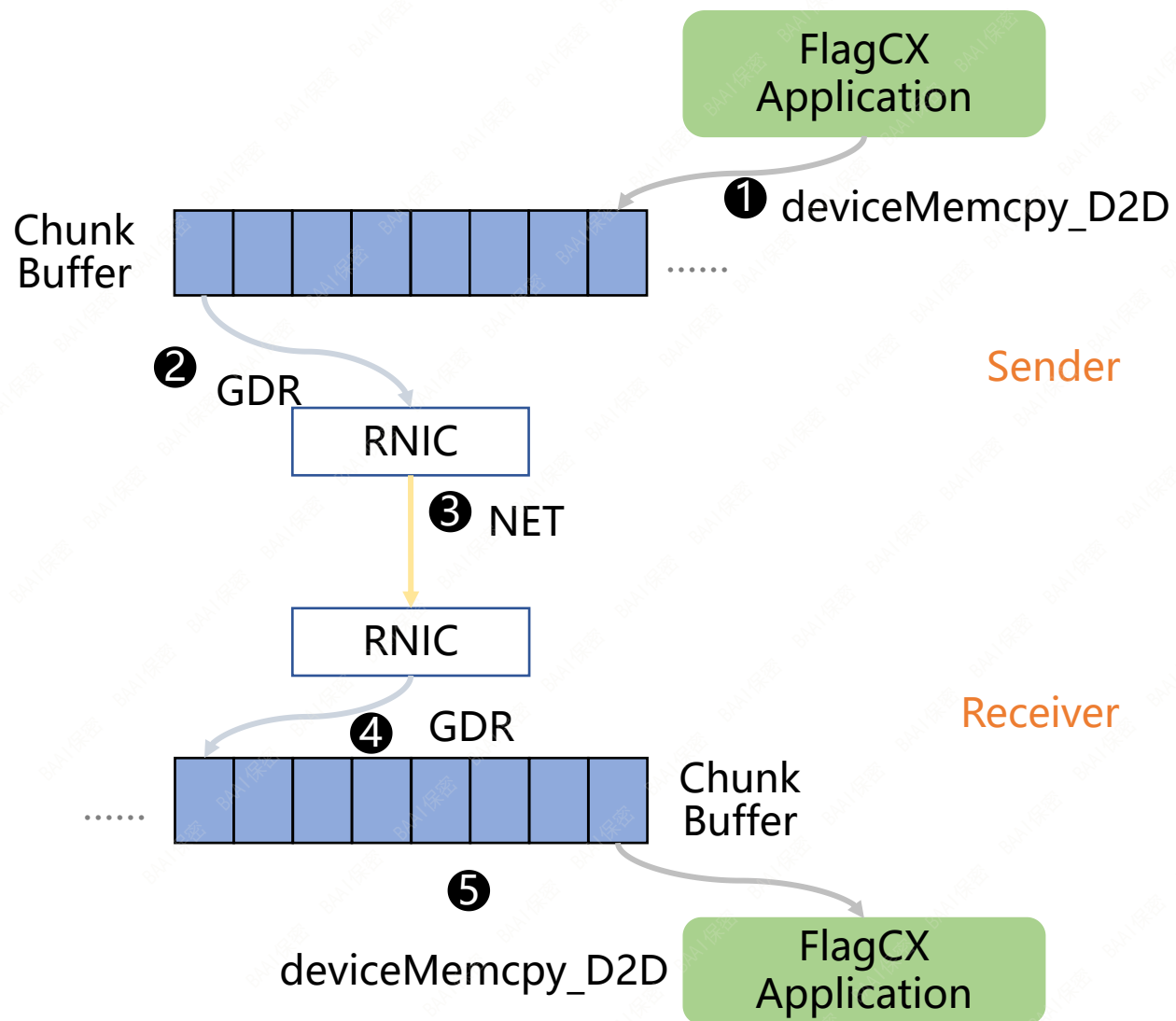
```
flagcxResult_t flagcxProxySend(sendNetResources *resources, void *data,
deviceAdaptor->deviceMemcpy(
    args->subs[step].stepBuff, (char *)data + args->totalCopySize,
    args->subs[step].stepSize, flagcxMemcpyDeviceToDevice,
    resources->cpStream, args->subs[step].copyArgs);
deviceAdaptor->eventRecord(resources->cpEvents[step],
    resources->cpStream); D2D
args->totalCopySize += args->subs[step].stepSize;
args->waitCopy++;
}

if (args->copied < args->waitCopy) {
    int step = args->copied & stepMask;
    if (deviceAdaptor->eventQuery(resources->cpEvents[step]) ==
        flagcxSuccess) {
        args->copied++;
    }
}

if (args->posted < args->copied) {
    void *req = NULL; 网络发送
    flagcxNetIb.isend(resources->netSendComm,
        args->subs[args->posted & stepMask].stepBuff,
        args->subs[args->posted & stepMask].stepSize, 0,
        resources->mhandles[0], &req);
    if (req) {
        args->subs[args->posted++ & stepMask].requests[0] = req;
    }
}

if (args->transmitted < args->posted) {
    void *req = args->subs[args->transmitted & stepMask].requests[0];
    int done = 0, sizes;
    flagcxNetIb.test(req, &done, &sizes);
    if (done) { 检测是否传输完毕
        args->transmitted++;
    }
} else {
    __atomic_store_n(args->hlArgs, 1, __ATOMIC_RELAXED);
    args->done = true;
    if (deviceKernel) { D2D
        deviceAdaptor->deviceMemcpy(args->dlArgs, args->hlArgs, 1,
```

Device-buffer RDMA – 内存注册



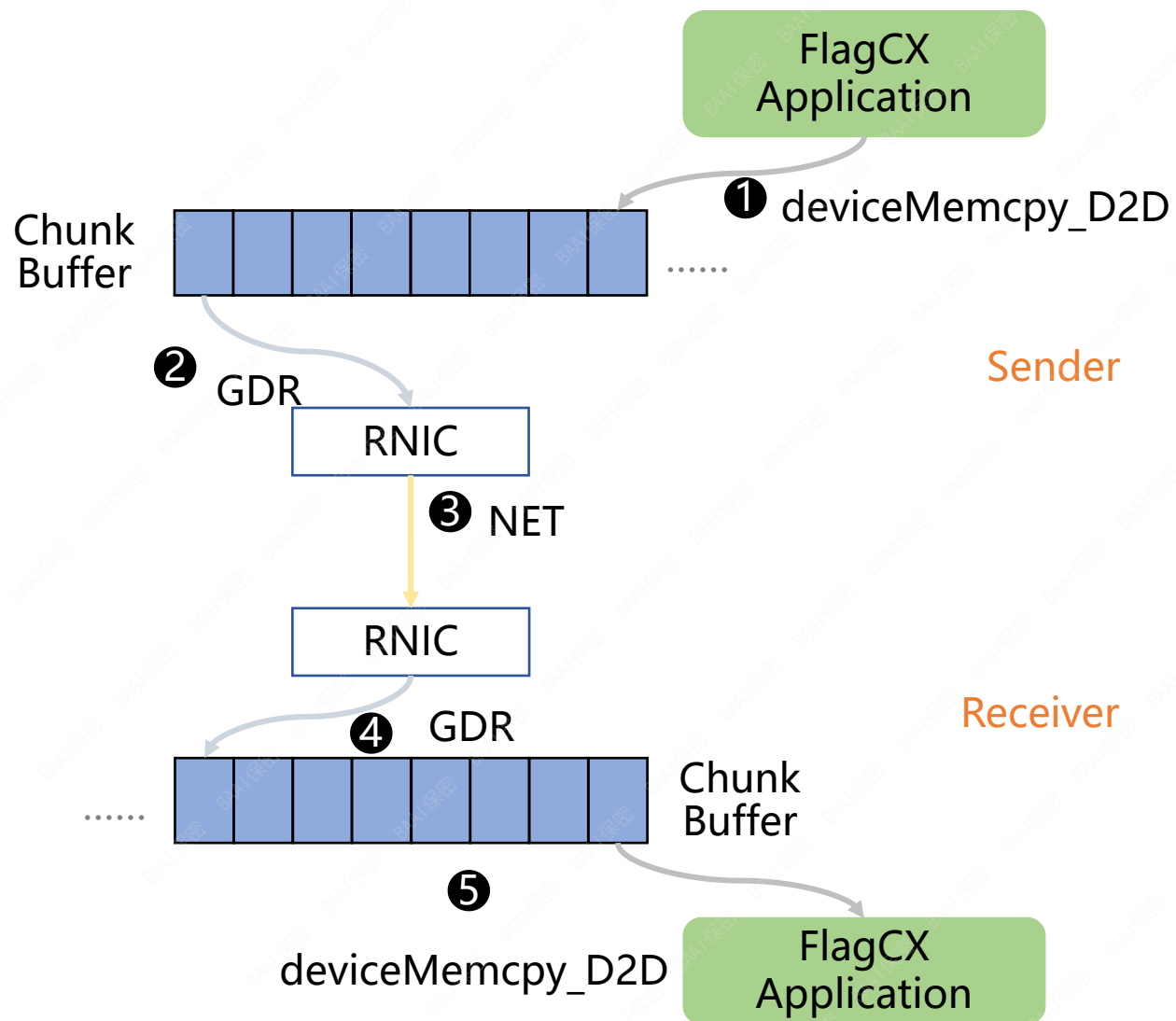
Peermem support:

1. gdrMemAlloc
2. ibv_reg_mr

DMA-BUF support:

1. deviceMalloc
2. getHandleForAddressRange
3. ibv_reg_dmabuff_mr

Device-buffer RDMA – 内存注册



```
flagcxResult_t cudaAdaptorGdrMemAlloc(void **ptr, size_t size,
                                       void *memHandle) {
    if (ptr == NULL) {
        return flagcxInvalidArgument;
    }
    DEVCHECK(cudaMalloc(ptr, size));
    cudaPointerAttributes attrs;
    DEVCHECK(cudaPointerGetAttributes(&attrs, *ptr));
    unsigned flags = 1;
    DEVCHECK(cuPointerSetAttribute(&flags, CU_POINTER_ATTRIBUTE_SYNC_MEMOPS,
                                  (CUdeviceptr)attrs.devicePointer));
    return flagcxSuccess;
}
```

```
flagcxResult_t
cudaAdaptorMemGetHandleForAddressRange(void *handleOut, void *buffer,
                                       size_t size, unsigned long long flags) {
    CUdeviceptr dptr = (CUdeviceptr)buffer;
    DEVCHECK(cuMemGetHandleForAddressRange(
        handleOut, dptr, size, CU_MEM_RANGE_HANDLE_TYPE_DMA_BUF_FD, flags));
    return flagcxSuccess;
}
```

Device-buffer RDMA – 总结

1. CCLAdaptor + DeviceAdaptor + TransportAdaptor

提供**多芯+多传输支持**



高可迁移性

一键编译多芯支持（8+）

2. CCLAdaptor + DeviceAdaptor + TransportAdaptor

+ Device-buffer RDMA提供**跨芯P2P支持**

算法设计

- Cluster分层

算法优化

- 拓扑探测
- 深度优化
- 代价建模

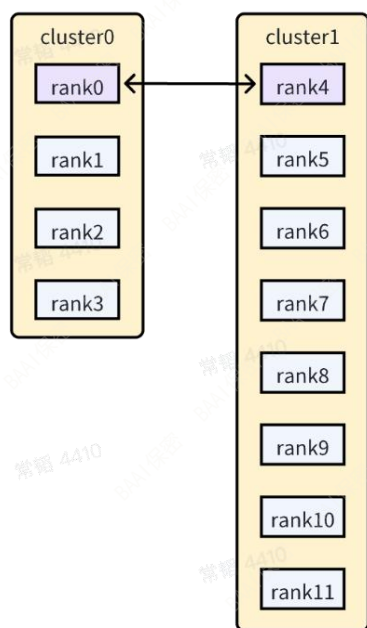
Cluster定义: 任意大小的同构设备组

跨芯集合通信算法的难点

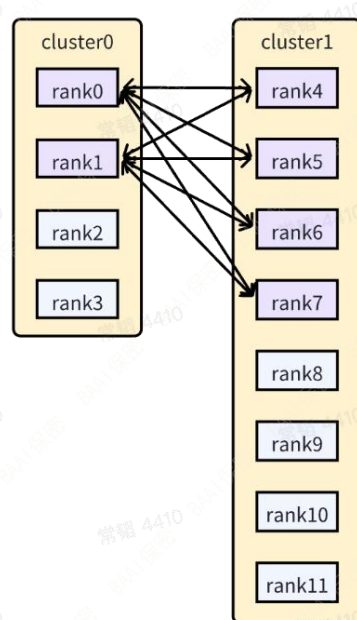
已知任一**Cluster**可以具备相同或不同的拓扑构型（比如网卡数），如何在保证跨芯集合通信算法**正确性**和**可扩展性**的同时**最大化通信带宽**？

正确性和可扩展性 →

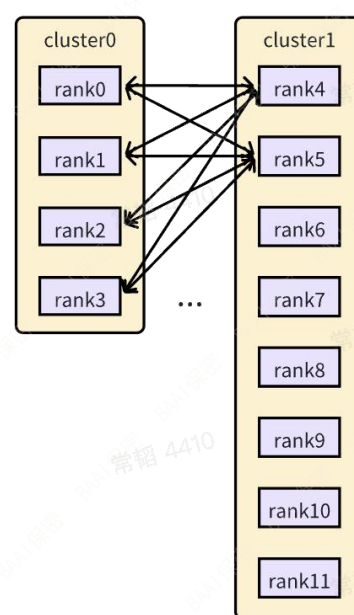
跨芯C2C算法



Mode 1. single-nic



Mode 2. normal multi-nic



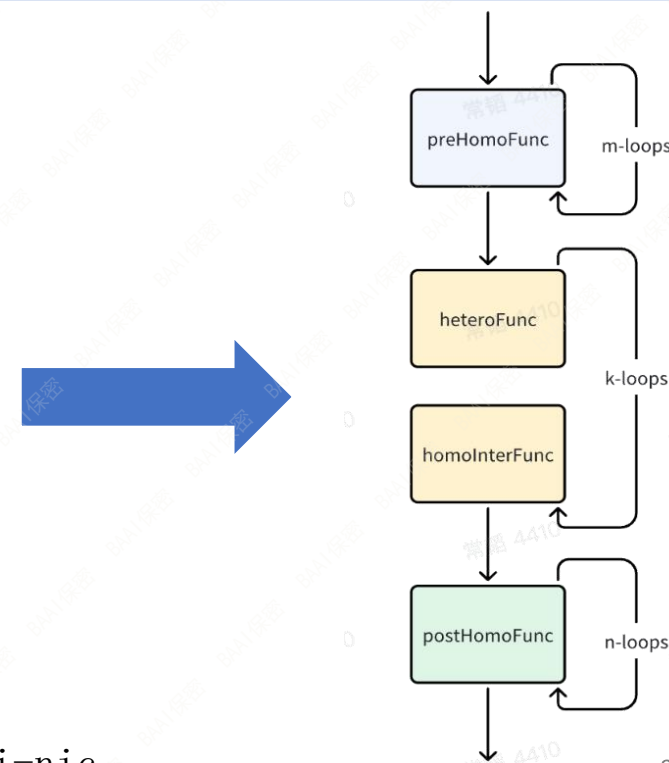
Mode 3. rank-local multi-nic

C2C AllReduce:

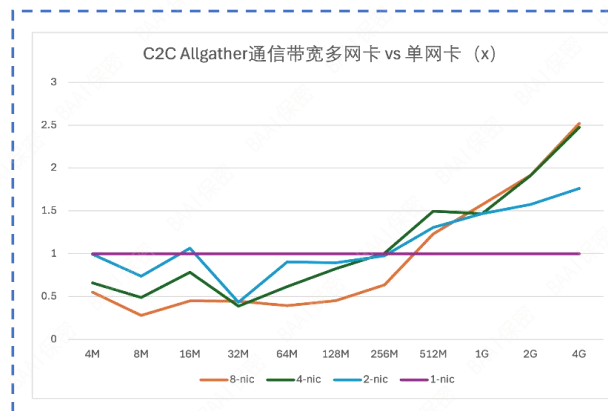
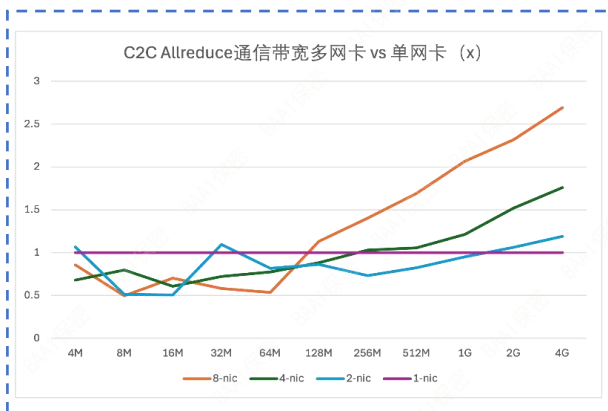
- * rank-local multi-nic, 同构ReduceScatter+跨芯Send/Recv+同构AllReduce
- * others, 同构Reduce+跨芯Send/Recv+同构AllReduce

C2C AllGather:

- * rank-local multi-nic, 同构AllGather+跨芯Send/Recv+同构Broadcast
- * others, 同构Gather+跨芯Send/Recv+同构Broadcast

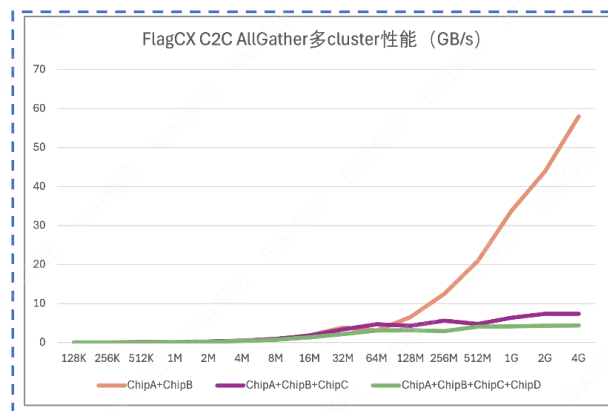
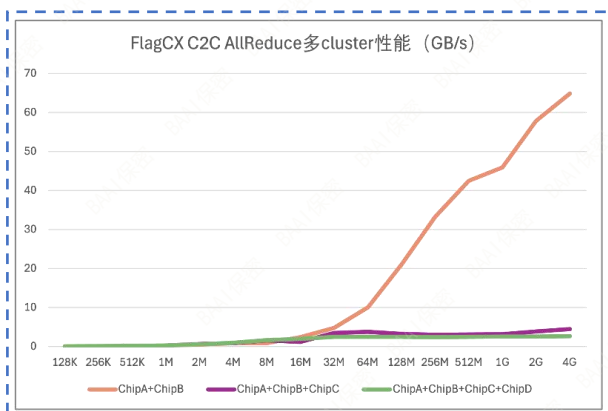


多网卡支持，可通过环境变量FLAGCX_IB_HCA设置网卡



- ❑ C2C AllReduce, 8-nic在128M以上通信量时超越单网卡 (1-NIC) 配置，而当通信量达到4GB时提升2.7倍
- ❑ C2C AllGather, 8-nic在384M以上通信量时超越单网卡 (1-NIC) 配置，而当通信量达到4GB时提升2.5倍

扩展性支持，可通过设置环境变量FLAGCX_CLUSTER_SPLIT_LIST切分sub-clusters



- ❑ FlagCX C2C算法理论上支持继续扩充cluster数量，已实现4-cluster跨芯通信
- ❑ 左边两张图中C2C AllReduce/AllGather的实测带宽会被引入的ChipC和ChipD限制，导致性能大幅下降

注：ChipC和ChipD是相同厂商的不同款芯片

Cluster定义： 任意大小的同构设备组

跨芯集合通信算法的难点

已知任一**Cluster**可以具备相同或不同的拓扑构型（比如网卡数），如何在保证跨芯集合通信算法**正确性**和**可扩展性**的同时**最大化通信带宽**？

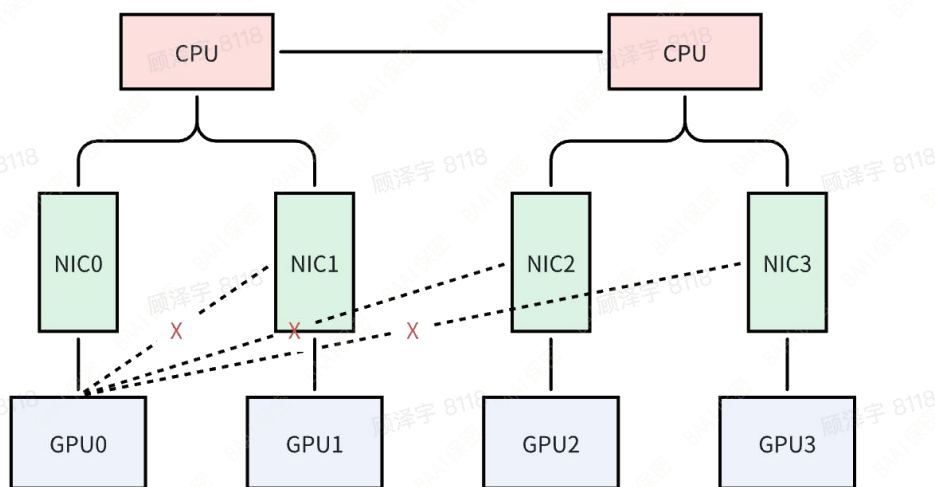
最大化通信带宽 → 自动拓扑探测获取最近网卡距离，把网卡距离最近的**rank**作为**inter-rank**参与跨芯通信

示例：

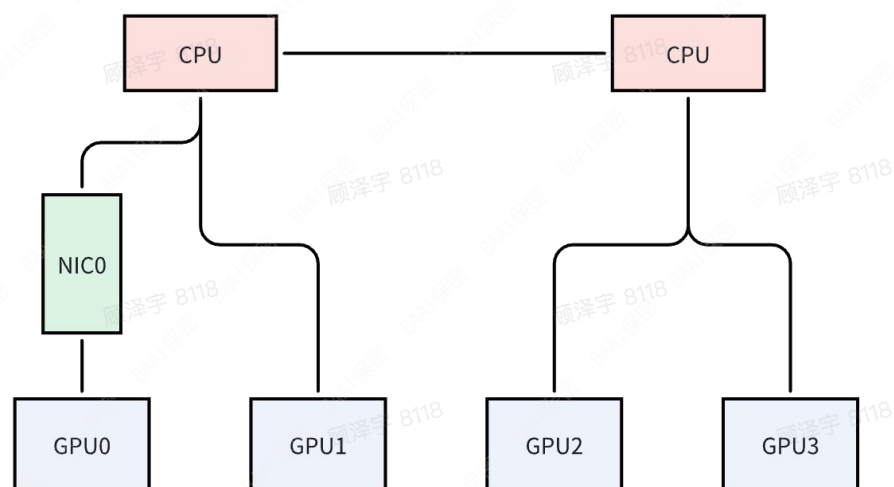
* cluster 0: 8gpu-8nic; cluster 1: 8gpu-8nic

* cluster 0: 8gpu-4nic; cluster 1: 8gpu-2nic

* cluster 0: 8gpu-8nic; cluster 1: 8gpu-8nic; cluster2: 16gpu-1nic



GPU0选择使用NIC0，而非其他距离较远的NIC



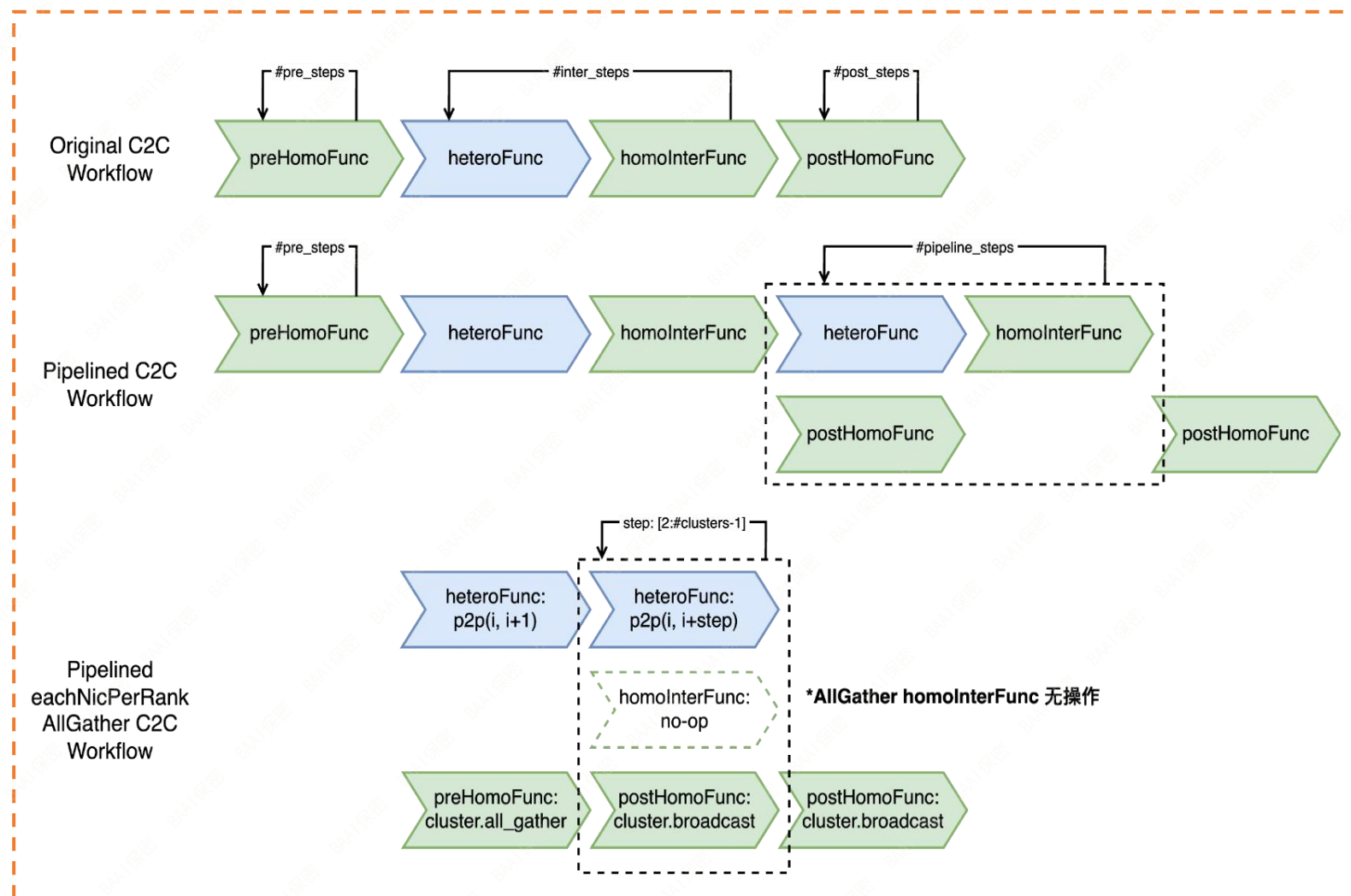
只使用GPU0进行跨芯通信可能比使用所有GPU更高效

主要瓶颈

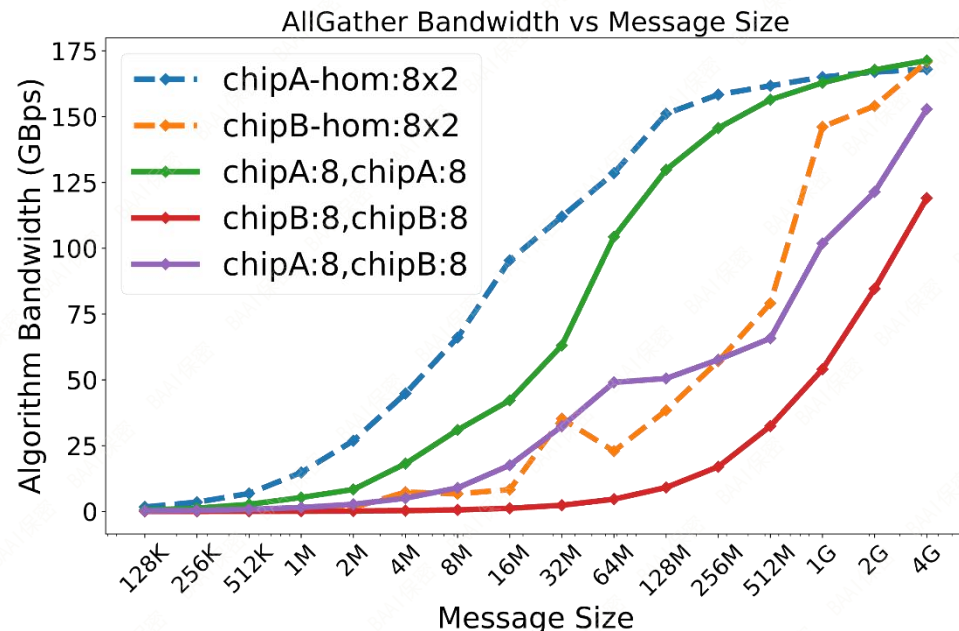
- 1) pre和post阶段可能涉及多轮，引入额外调用开销
- 2) pre、inter和post阶段串行处理，存在overlap空间
- 3) 跨芯sendrecv负载不均
- 4) ...

优化方案

- 1) pre和post阶段添加group操作
- 2) 流水线并行
- 3) 跨芯sendrecv负载均衡策略搜索
- 4) ...



最佳性能实践 – AllGather

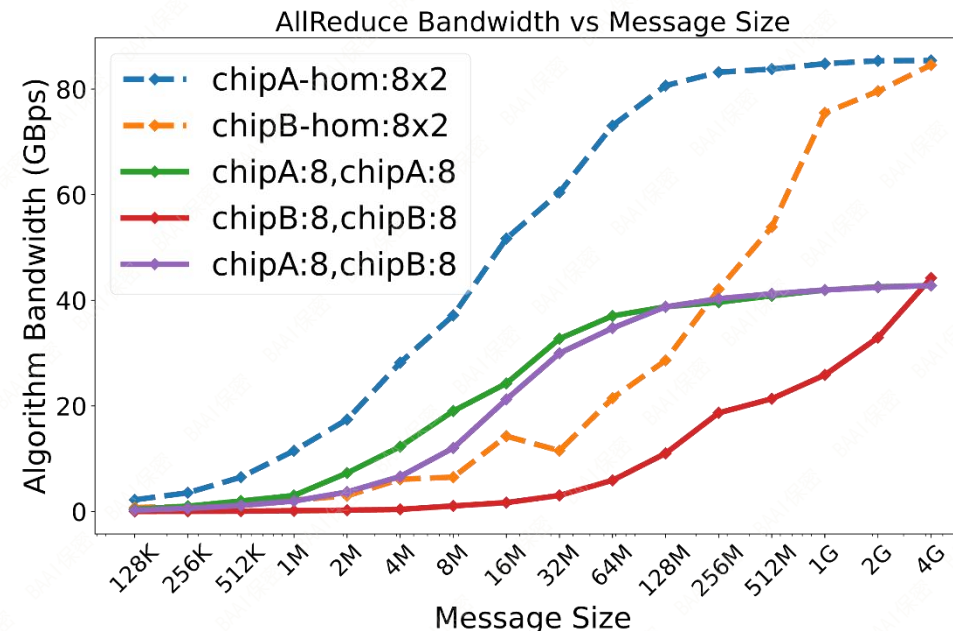


ChipA 2机, Hetero在大Message Size上 ($\geq 128M$) 性能接近Homo的**100%**

ChipB 2机, Hetero在大Message Size上性能接近Homo的**70%**

Chip A + ChipB 2机, 大Message Size上性能接近Chip A/B 2机Homo的**90%**

最佳性能实践 – AllReduce



ChipA 2机, Hetero在大Message Size上性能接近Homo的**50%**

ChipB 2机, Hetero在大Message Size上性能接近Homo的**50%**

Chip A + ChipB 2机, 大Message Size上性能接近Chip A/B 2机Homo的**50%**

问题：给定多个clusters、多种拓扑构型、多种优化技术，如何选取最优跨芯通信策略？

筛选最优跨芯通信策略 ->

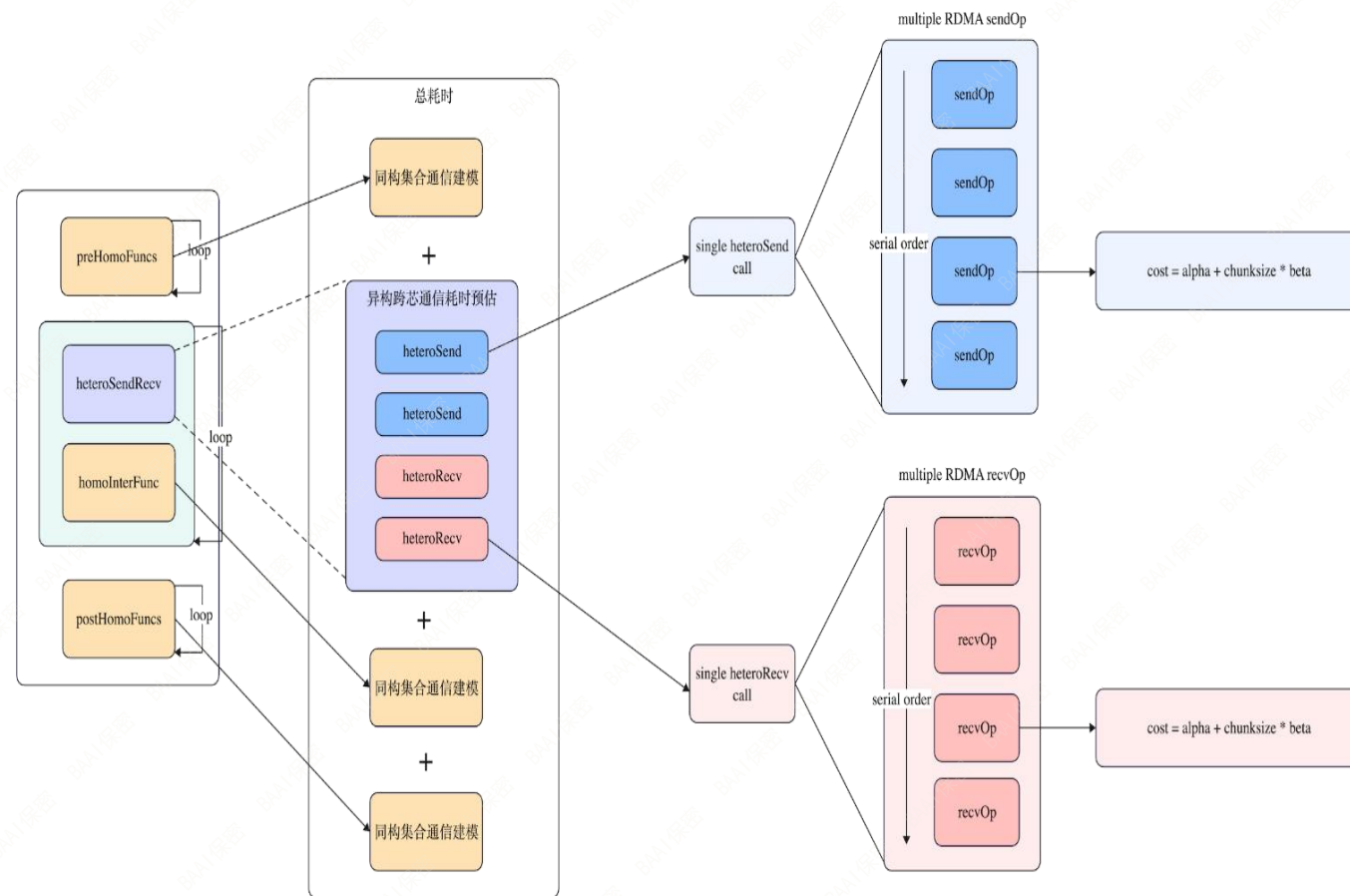
代价建模

代价建模能力：

- › 支持集合通信算法端到端耗时建模
- › 结合拓扑信息及不同厂商性能差异，做精细化耗时预估
- › 提供统一接口，供算法模块调用，自动选择最优策略

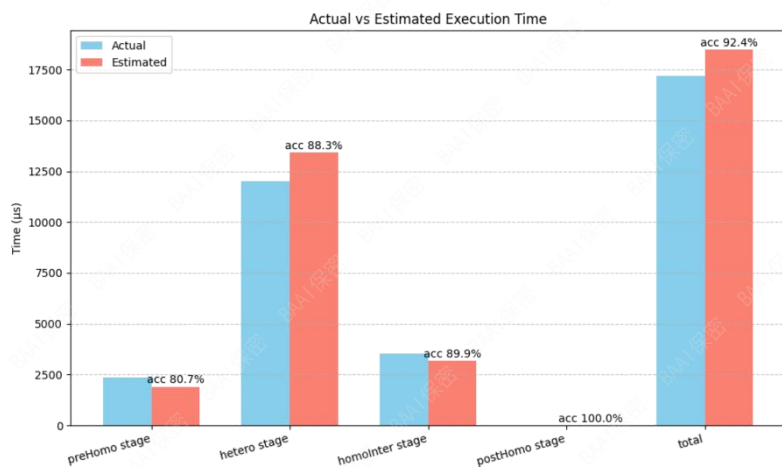
代价建模设计要点：

- › 输入参数包括pipeline数，网卡数等
- › 采用 α - β 模型
- › 对算法不同阶段进行耗时预估
- › 通过厂商硬件时延和拓扑模块计算出的带宽信息进行计算



(上图引用顾泽宇-拓扑和代价建模PPT)

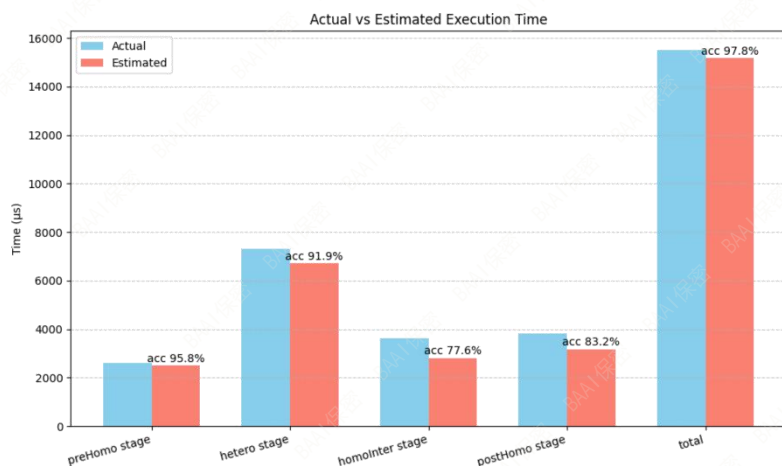
各阶段开销预估数据 vs 实测数据



ChipA 2机Hetero

1-nic : 1-nic

Acc: 80.7% - 92.4%

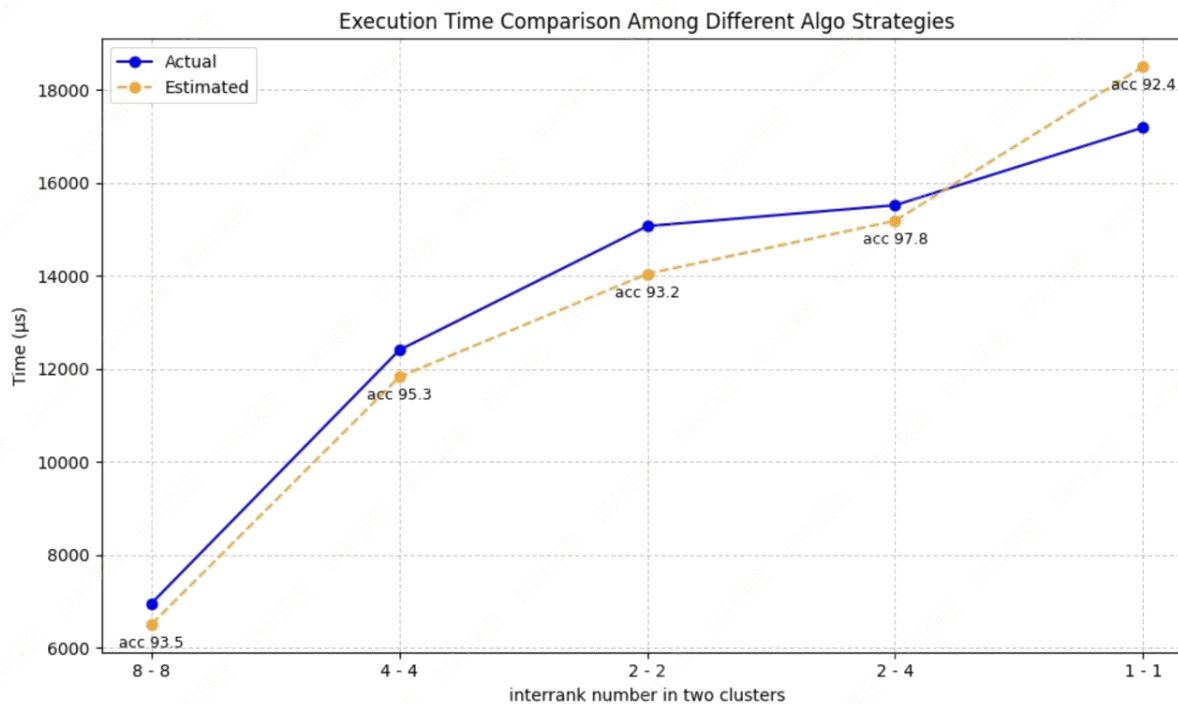


ChipA 2机Hetero

2-nic : 4-nic

Acc: 77.6% - 95.8%

端到端开销预估数据 vs 实测数据



ChipA 2机Hetero, 不同网卡配置, 端到端开销Acc: 92.4% - 97.8%

Cluster-to-Cluster算法 – 总结

1. CCLAdaptor + DeviceAdaptor + TransportAdaptor

提供**多芯+多传输支持**



高可迁移性

一键编译多芯支持（**8+**）

2. CCLAdaptor + DeviceAdaptor + TransportAdaptor

+ Device-buffer RDMA提供**跨芯P2P支持**



高扩展性

任意扩展多cluster通信（**4+**）

3. CCLAdaptor + DeviceAdaptor + TransportAdaptor

+ Device-buffer RDMA + C2C算法提供**跨芯Collective**

支持



高性能

C2C算法性能可接近原生通信库（**90+%**）

一些想法

1. CCLAdaptor + DeviceAdaptor + TransportAdaptor的设计范式和业界相关工作（比如msccl、deeplink等）正交，可探索实现协同作业
2. Cluster根据划分粒度（比如单机、多机、超节点、跨DC等）的选择，可以进行不同方向的研究。其核心是屏蔽Cluster内部细节，专注于跨Cluster通信（网络）
3. C2C算法当前仅用于跨芯集合通信，后续会基于此研究跨芯通信的自动编译技术
4. C2C算法可扩展到任意能够被拆解为Cluster内和Cluster间执行逻辑的实际通信任务，比如异构PD分离等

欢迎各位同学贡献FlagOpen开源社区！

FlagCX: [*https://github.com/FlagOpen/FlagCX*](https://github.com/FlagOpen/FlagCX)

FlagScale: [*https://github.com/FlagOpen/FlagScale*](https://github.com/FlagOpen/FlagScale)



联系我们

邮件: job@baai.ac.cn

电话: 010 - 6893 3383

地址: 北京市海淀区成府路150号智源大厦



智源公众号

1. CCLAdaptor + DeviceAdaptor提供**多芯支持**
2. CCLAdaptor + DeviceAdaptor + TransportAdaptor提供**跨芯P2P支持**
3. CCLAdaptor + DeviceAdaptor + TransportAdaptor + C2C算法提供**跨芯Collective支持**



高可迁移性

一键编译多芯支持 (**8+**)



高扩展性

任意扩展多cluster通信 (**4+**)



高性能

C2C算法性能可接近
原生通信库 (**90+%**)